QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 6
5. Test Case Specifications 8
6. AI Output & Boundary Testing 10

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - AI Spending Decision Assistant
Team Repo URL	GitHub Repo - Stratatouille
Project Board URL	Project Board - Trello
Live Deployment URL (optional for preliminary round)	Not available for preliminary round

Objective:

The objective of this testing process is to ensure that the AI Spending Decision Assistant provides reliable financial insights, maintains system stability under different scenarios, and handles AI service failures gracefully through fallback mechanisms. The system must consistently return valid responses for all supported features, including AI chat, affordability checking, and expense tracking.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **AI Chat:** Provides financial advice based on user expenses
- **Affordability Checker:** Evaluates whether a user can afford a specific purchase
- **Expense Tracking:** Records and stores user spending data in Firestore
- **Auto Categorization:** Assigns categories to expenses
- **Explainability:** Outputs responses in Answer / Reason / BasedOn format

1.2 Out-of-Scope

- User authentication and profile management
- External financial integrations (bank APIs)
- Advanced financial forecasting or prediction models

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI API timeout/ instability	5	4	20	Implement fallback logic to ensure consistent response	Simulate timeout and verify fallback response
Incorrect financial recommendation	3	4	12	Use deterministic logic for key flows	Test with known expense inputs
Firestore data inconsistency	3	3	9	Use single source of truth for expenses	Restart system and verify data persistence
Slow response time	4	3	12	Fine tune timeout (12s backend)	Measure response time manually

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
    - **Scope:** Core logic such as affordability calculation, expense handling, and fallback logic.
    - **Execution:** Tests are performed manually during development using the application interface and backend logs.
    - **Isolation:** Core logic is tested independently without relying on external AI responses by triggering fallback scenarios.
    - **Pass Condition:** System returns correct outputs for both valid inputs and edge cases without crashing.

- **Integration Test**
    - **Scope**: Interaction between Flutter frontend, FastAPI backend, Firestore database, and GLM API.
    - **Execution:** Performed after integrating frontend and backend components to ensure full data flow works correctly.
    - **Workflow:** Real API calls and Firestore reads/writes are tested without mock data to ensure real-world behavior.
    - **Pass Condition:** All components communicate correctly and return expected responses.

- **Test Environment (CI/CD Practice):**
    - **Local Testing:** Testing is performed manually on localhost using Flutter Web and FastAPI backend.
    - **Staging/CI:** Not implemented for the preliminary round.
    - **CI/CD automated pipeline:** Not implemented; testing is conducted manually due to time constraints.

- **Regression Testing & Pass/Fail Rules:**
    - **Execution Phase**: Testing is conducted after major updates to ensure all core features remain functional.
    - **Pass/Fail Condition:** A test is considered passed only if the actual output matches the expected result.
    - **Continuation Rule**: Critical test cases must pass before proceeding to further testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Data Strategy:**
    - **Manual:** Test data is generated by adding expenses directly through the application interface which includes food, transport, bills, and miscellaneous expenses, to simulate realistic user behavior. Edge cases are also included, such as empty expense lists and unusual inputs, to ensure the system handles unexpected scenarios properly.
    - **Automated:** Not implemented due to time constraints. However, the system uses consistent data structures (Firestore collections) to ensure reliability during repeated testing.
- **Passing Rate Threshold:**
    - A minimum of 80% of all test cases must pass for the system to be considered stable. For critical test cases (e.g., AI response handling, fallback reliability, and affordability calculation), a 100% pass rate is required. Any failure in critical cases must be resolved before proceeding. This ensures that even if non-critical features have minor issues, the core functionality of the system remains reliable and usable.

4. CI/CD Release Thresholds & Automation Gates

This section defines the criteria required before code changes are accepted into the main branch or considered ready for deployment. These checks ensure that the system remains stable and functional for demonstration purposes.

4.1 Integration Thresholds (Merging to Main)

These checks ensure that any updates merged into the main branch do not break core functionality. The main branch is treated as a stable version of the system for testing and demonstration.

Checks	***Requirements***	***Project***	***Pass/Failed***
Backend Build / Startup	FastAPI starts without import errors	Passed	Passed
Flutter Web Build / Run	App loads successfully in Chrome	Passed	Passed
Core Feature Regression	AI Chat, Add Expense, Dashboard, and Affordability Checker remain functional	Passed	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

Code Quality	No demo-breaking runtime errors	Passed with minor known limitations	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	Core demo flows must pass	Passed	Passed
AI Output Pass Rate	System must return either GLM or fallback response	Passed	Passed
Critical Bugs	Zero known demo-blocking bugs	0 demo-blocking bugs	Passed
API Performance	Response should return within backend timeout threshold	12s backend timeout with fallback	Passed
Security	API keys are not exposed	Backend environment / config based	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications

This section presents key test cases used to validate the system's core functionality, including normal usage, failure scenarios, and system reliability under different conditions.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** Expense Tracking + AI Decision Support	Verify that a user can add an expense, view it in recent history, and receive financial advice based on the updated spending data.	1. Open the Flutter Web application. 2. Go to Add Expense. 3. Add an expense such as "coffee", RM8, Food. 4. Confirm the expense appears in Recent History. 5. Go to AI Chat. 6. Ask "Can I afford RM20?"	Expense is saved successfully and the system returns an affordability response with Answer, Reason, and BasedOn.	Expense was saved and the system returned a valid affordability response. Status: Passed
TC-02	**Specific Case (Negative):** AI API Timeout / Failure	Verify that the system still provides a usable response when the AI service is slow or unavailable.	1. Ask a financial advice question in AI Chat. 2. Simulate or observe AI timeout due to unstable API response. 3. Check whether fallback response is returned.	The system should not crash or remain in constant loading state. A fallback response should be returned using available expense and budget data.	Fallback response was returned successfully with explanation. Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-03	**NFR (Performance):** AI Response Time	Verify that the AI Chat does not wait indefinitely for external AI service response.	1. Ask “Why am I overspending?” in AI Chat. 2. Observe whether GLM response or fallback response is returned. 3. Confirm response is returned within the configured timeout threshold.	System returns either AI or fallback response within approximately 12–14 seconds.	System returned a response within the acceptable timeout range. Status: Passed
TC-04	**NFR (Reliability):** Firestore Data Persistence	Verify that expense data remains available after refreshing or reopening the application.	1. Add a new expense through the Add Expense screen. 2. Refresh or reopen the web application. 3. Check Recent History and Dashboard.	The expense remains visible and budget calculations remain updated.	Expense persisted and was reflected in Recent History and Dashboard. Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing

This section evaluates the AI component to ensure that it produces acceptable and reliable outputs, and that the system handles abnormal or unexpected inputs safely.

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	What category should I reduce?	Recommends the highest spending category and explains using recent expense data.	Recommended a category using either GLM or fallback logic.	Passed
AI-02	Can I afford RM157?	Gives an affordability decision with Answer, Reason, and BasedOn.	Returned affordability response with explanation.	Passed
AI-03	What are my 4 most recent spendings?	Lists recent expenses accurately based on stored expense data.	Returned latest expenses from app data.	Passed

6.2. Oversized/Larger Input Test

This section tests how the system behaves when handling longer or more complex user inputs, and whether it remains stable under such conditions.

Fields	Details
Maximum Input Size	Short text-based queries (approximately 1–2 sentences) with a limited list of recent expenses passed to the backend.
Input used while testing	A long, detailed financial question including multiple expenses and explanations (approximately 4–5 sentences).

© UMHackathon 2026

Email: umhackathon@um.edu.my

Expected Behavior	The system should process the input and return either an AI-generated response or a fallback response within the configured timeout. If the input is too complex or takes too long to process, the fallback mechanism should ensure a valid response is still returned.
Actual Behavior	For longer or more complex inputs, the GLM response may time out due to API instability. In such cases, the system successfully triggers fallback logic and returns a valid response based on available expense and budget data.
Status	Passed

6.3. Adversarial/Edge Prompt Test

Prompt Input:
"asdfasdfasdf can spend or not???"

Expected Behavior:
The AI may not fully understand the malformed or ambiguous input. The system should not crash and should still return a safe and structured response.

Actual Behavior:
The GLM response was inconsistent due to unclear input. In some cases, it produced a vague response, while in others it failed to respond within the timeout. The system handled this by triggering fallback logic, which returned a structured response based on available expense and budget data.

System Handling:
The system ensures stability by not relying solely on AI understanding. When input is ambiguous or malformed, fallback logic guarantees a valid response is returned, preventing system failure or undefined behavior.

6.4. Hallucinating Handling

- The system mitigates hallucination primarily through controlled input context and structured output requirements. Only relevant financial data, such as recent expenses and remaining budget, is passed to the AI model to reduce the likelihood of fabricated responses.

- For queries that require exact data (e.g., recent spending), deterministic logic is used instead of relying entirely on AI outputs. This ensures factual accuracy for critical information.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Additionally, the system enforces a structured response format consisting of Answer, Reason, and BasedOn, which helps constrain the AI output and improve interpretability.

- In cases where the AI response is delayed, inconsistent, or unavailable, fallback logic is triggered to generate a response directly from real expense data, further reducing the risk of hallucinated or misleading outputs.

Important Note:

Using Agile principles, testing is conducted iteratively throughout the development process rather than only at the end. This allows continuous validation of system behavior and ensures that issues are identified and addressed early during development.

© UMHackathon 2026

Email: umhackathon@um.edu.my